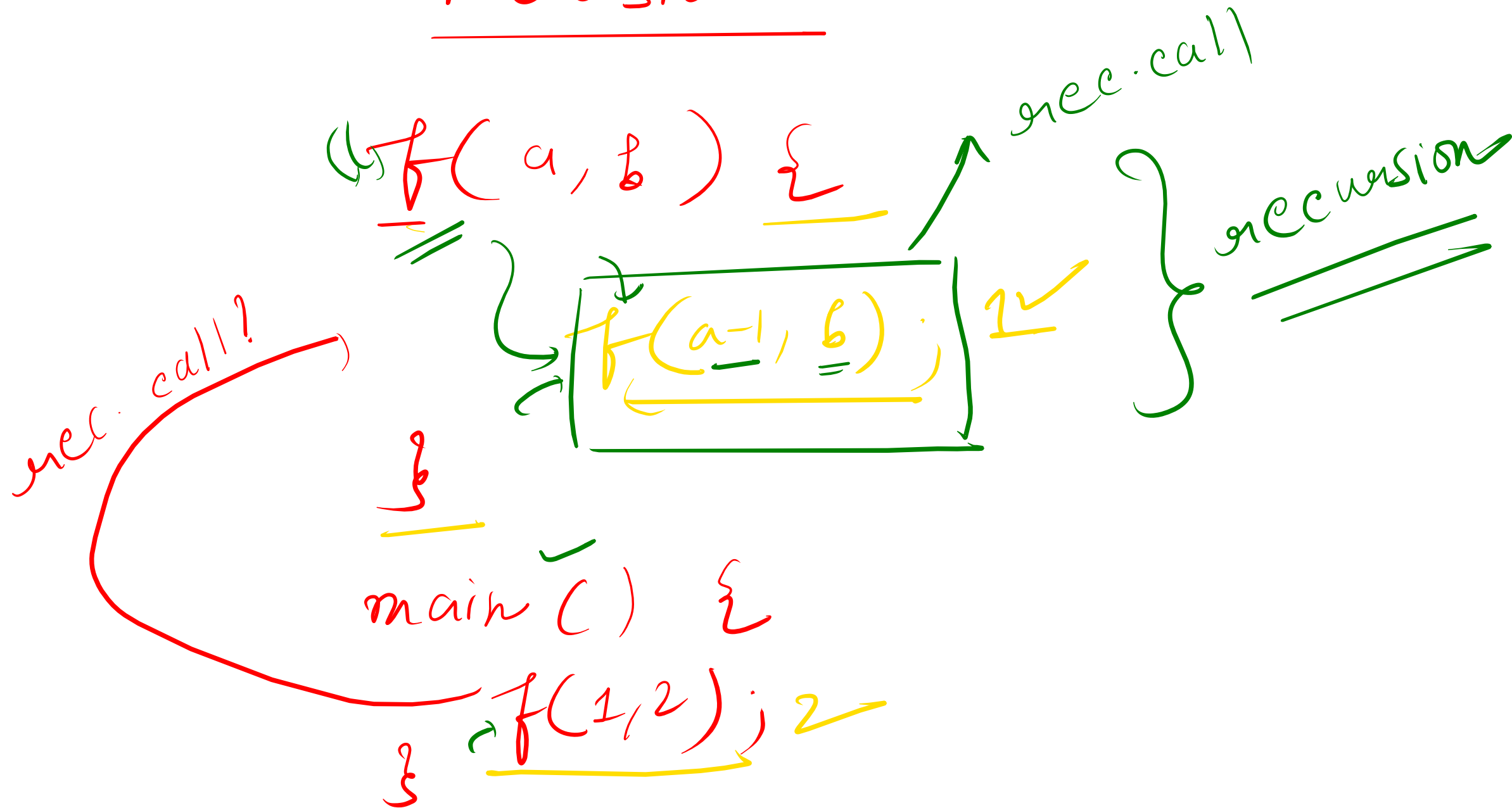


Recursion



when we require
that?

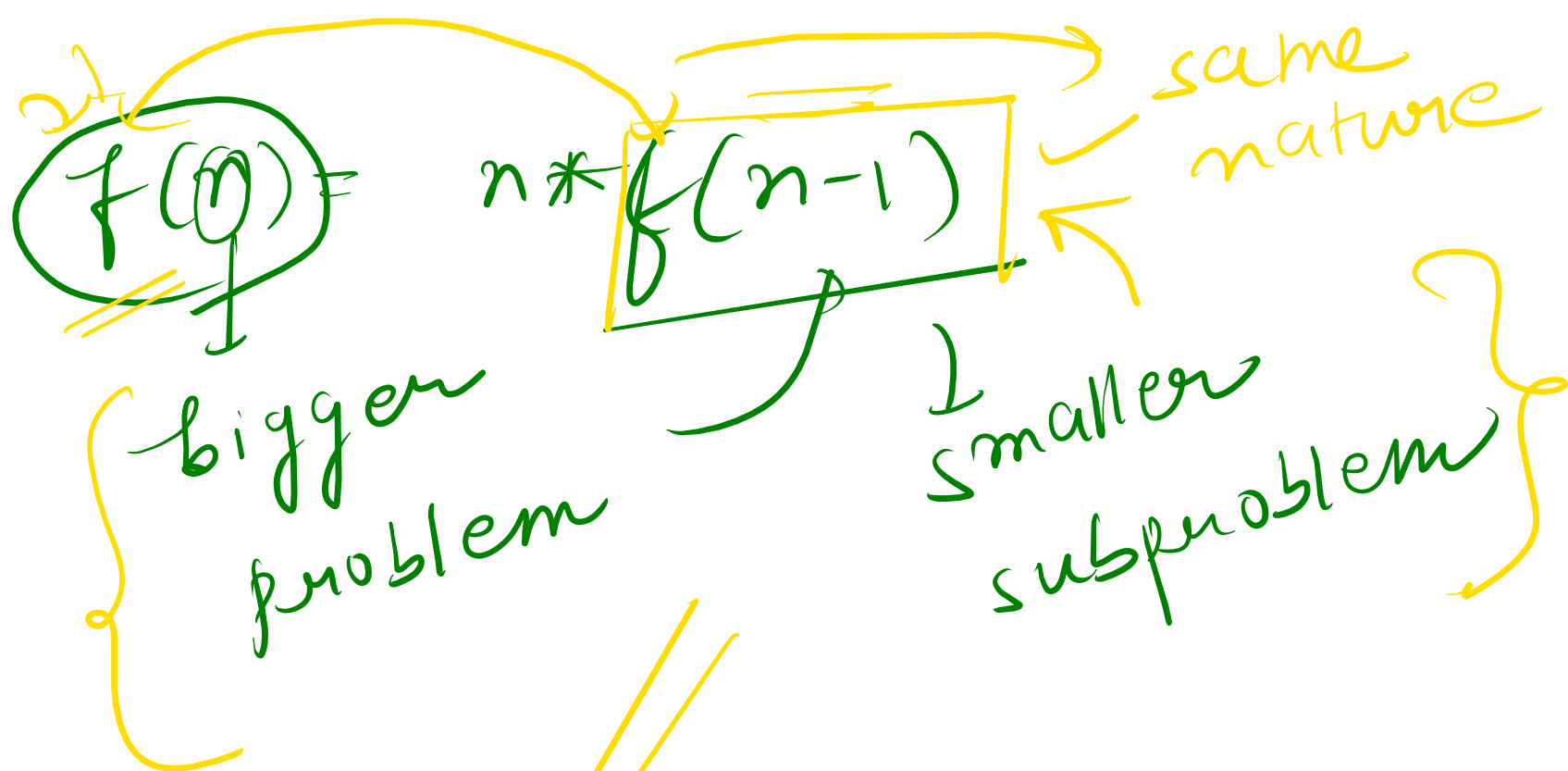
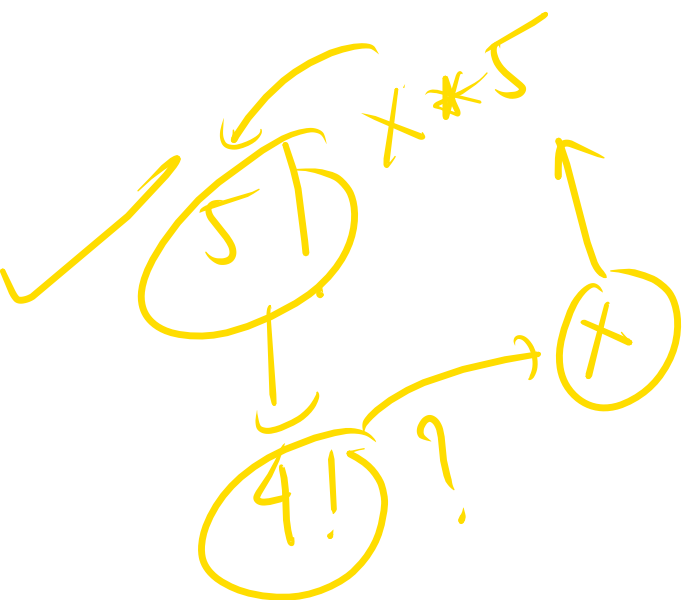
factorial

$$\begin{aligned} \underbrace{f(\underline{n})}_{\text{factorial}} &= \underbrace{n * (n-1 * n-2 * n-3 * n-4 * \dots * 1)}_{(n-1)!} \\ &\rightarrow \underline{n * f(n-1)} \end{aligned}$$

$\Downarrow$

$f(n-1)$

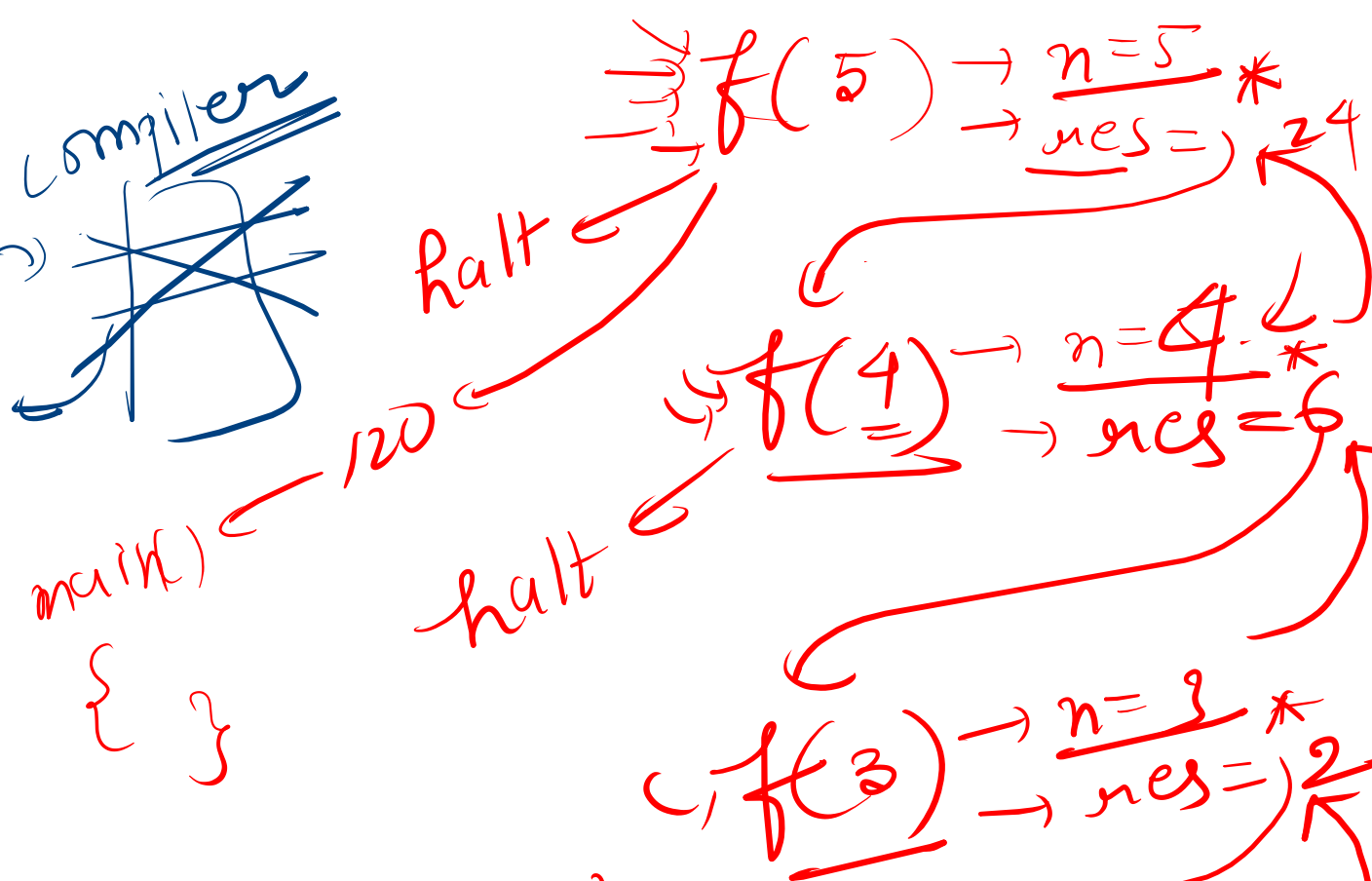
?



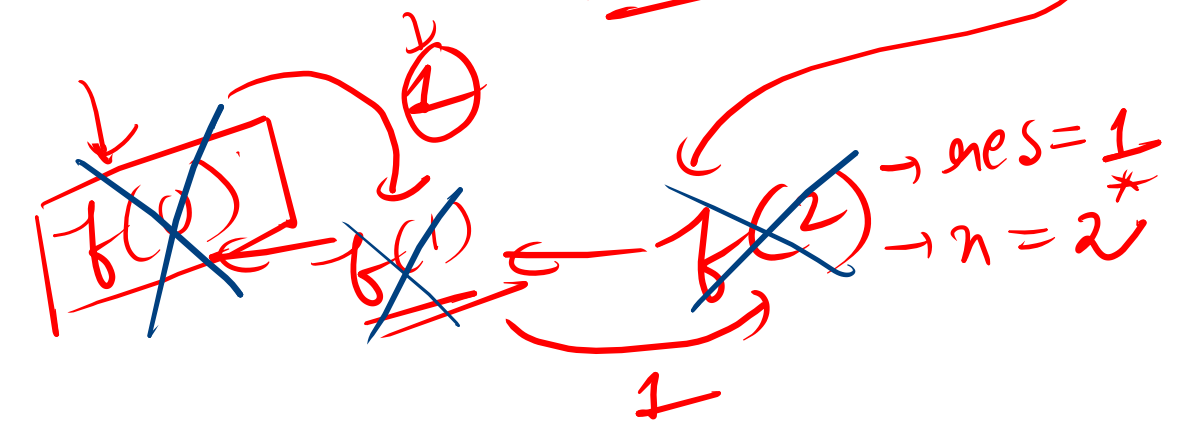
Recursive in nature.

$f(n)$ { ^{4 bytes}
 $\Rightarrow$ if $(n == 0)$ ^{base case} return 1;
 {
 int res = $f(n-1)$;
 return res * n;
 }

~~compiler~~
~~f~~



(TC) ^{4 bytes}
 8 bytes + extra
 n size $\times$
 $n == 0/1$



$f(5)$ ✓ $\rightarrow (n!) \times$

$n=5$

$f(4)$ ✓

$f(3)$

$f(2)$

$f(1)$

$f(0)$

$n+1$
↓
function calls

$n+1$
↓
full
exec.

c ✓

$O((n+1) \times \dots)$

$f(n) \{$

$\text{if}(n==0)$

$\text{return } 1;$

$\{ \text{mes} = f(n-1);$

$\text{return } n * \text{mes};$

$\}$

5/6/7

$\rightarrow c_1$

c_2

c_3

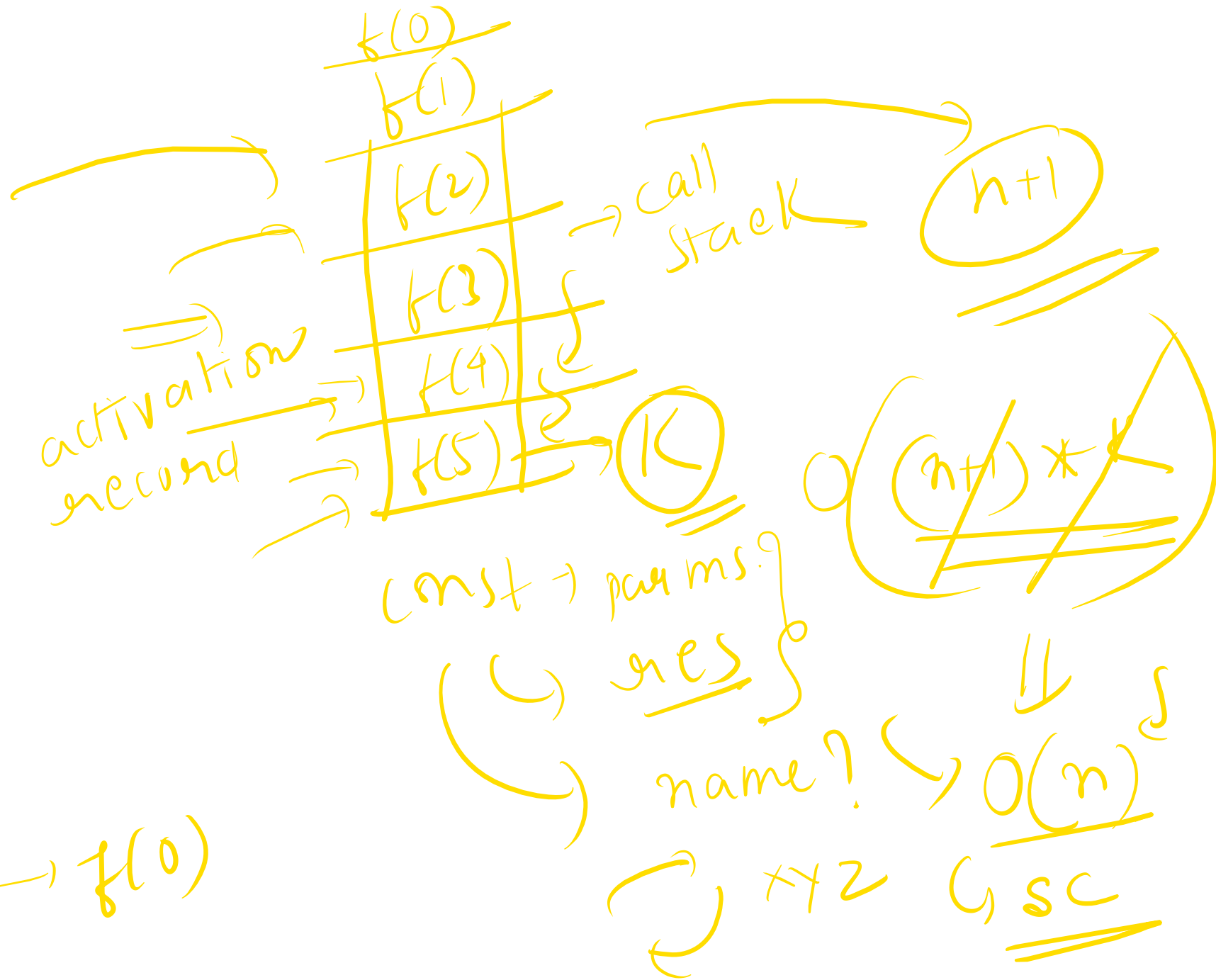
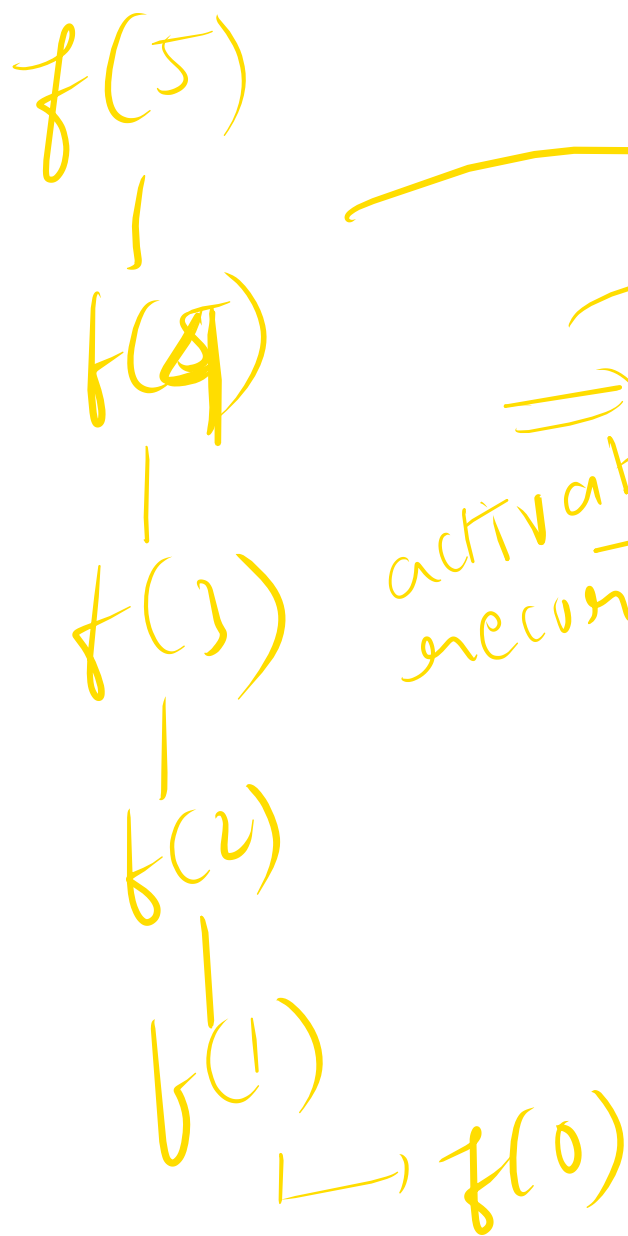
c

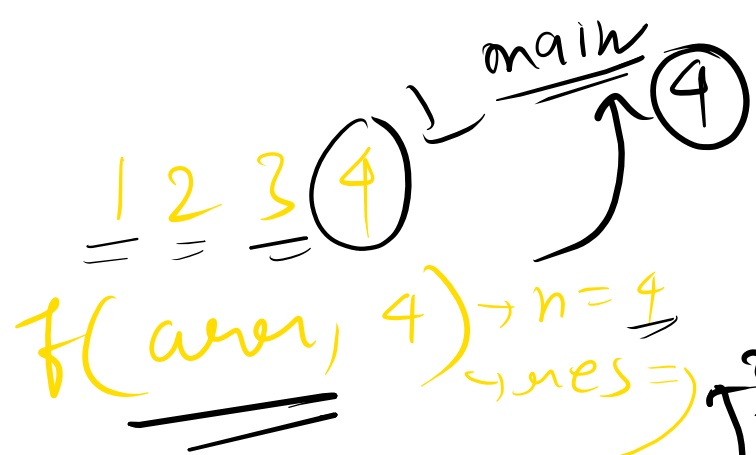
↓

$\{ c_1 + c_2 + c_3 \}$

c

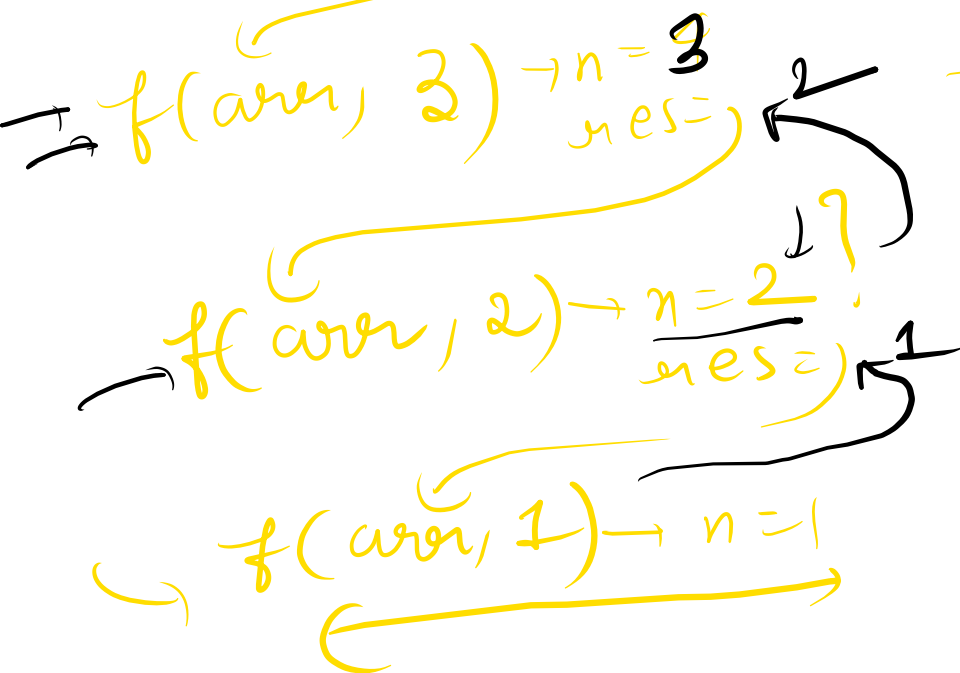
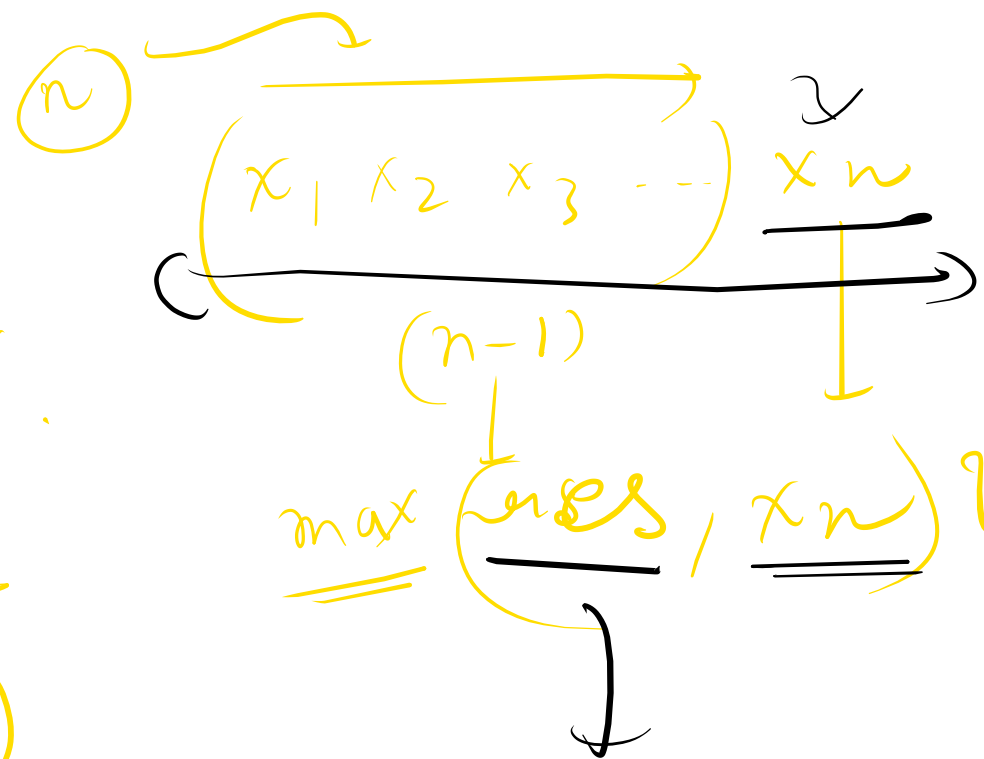
→ n





max of an array

recursively.



f(arr, n) {

if (n == 1)

return arr[0];

res = f(arr, n-1);

return max(arr[n-1], res); }